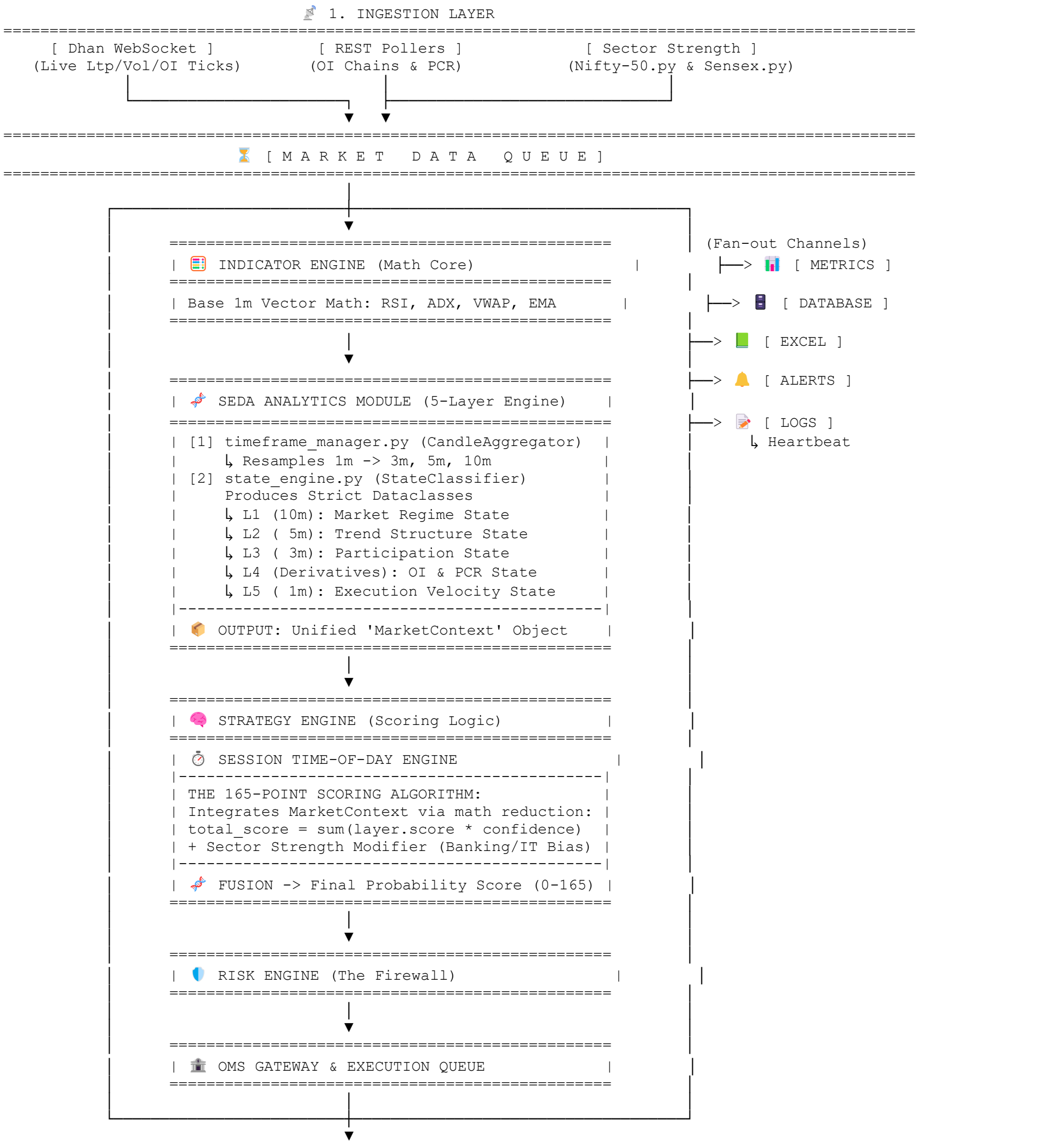
NEW MASTER ARCHITECTURE DIAGRAM (WITH SEDA 5-LAYER INTEGRATION)

Staged Event-Driven Architecture (SEDA) with Analytics Module Injection



The 5-Layer SEDA Architecture Details & Data Models

Instead of returning basic strings, the `state_engine.py` will encapsulate all evaluations into dedicated strict `Dataclasses`. All layers are then bundled into a master `MarketContext` payload.

Unified Base Properties for all States
Every state returned by layers 1, 2, 3, and 5 will inherit or utilize:
- `state: str` (e.g., "Bull Trend")
- `confidence: float` (e.g., 0.82)
- `score: int` (e.g., 13)

- `timestamp: datetime` (Ensures temporal alignment)

1. Market Regime (10m) -> `RegimeState`
- **Inputs**: EMA9 vs EMA20, ADX, Supertrend, ATR Expansion
- **Outputs**: `RegimeState(state="Strong Bull Trend", confidence=0.87, score=18, timestamp=...)`
- **Purpose**: Establishes the macro-level directional bias for the day.

2. Market Structure (5m) -> `StructureState`
- **Inputs**: Price Action (Higher Highs / Lower Lows), Swing Breaks, Range Width, Compression
- **Outputs**: `StructureState(state="HH-HL Expansion", confidence=0.75, score=15, timestamp=...)`
- **Purpose**: Validates if the price action confirms the 10m regime.

3. Participation Engine (3m) -> `ParticipationState`
- **Inputs**: VWAP, Volume Delta Proxy, Relative Volume, Delivery %
- **Outputs**: `ParticipationState(state="Institutional Buying", confidence=0.90, score=10, timestamp=...)`
- **Purpose**: Confirms that moves are backed by true market volume, filtering out low-liquidity fakeouts.

4. Derivatives Engine -> `DerivativeState`
- **Inputs**: OI Change, PCR, ATM OI Shift, OI Build-Up, IV Change
- **Model Output**:
  ```python
  @dataclass
  class DerivativeState:
      buildup_type: str  # e.g., "Long Build-Up", "Short Covering"
      pcr_bias: str      # e.g., "Bullish", "Bearish"
      oi_trend: str      # e.g., "Expanding", "Contracting"
      confidence: float  # e.g., 0.88
      score: int         # e.g., 20
      timestamp: datetime
  ```
- **Purpose**: Provides institutional options-market sentiment (the missing link in pure price-action strategies).

5. Execution Engine (1m) -> `ExecutionState`
- **Inputs**: RSI, ATR Velocity, Distance from VWAP, Volume Surge, Candle Spread
- **Outputs**: `ExecutionState(state="Accelerating", confidence=0.95, score=10, timestamp=...)`
- **Purpose**: Pinpoints the exact micro-second trigger for entry, maximizing RR ratio.

Master Data Object
```python
@dataclass
class MarketContext:
    regime_10m: RegimeState
    structure_5m: StructureState
    confirmation_3m: ParticipationState
    derivatives: DerivativeState
    execution_1m: ExecutionState
    context_timestamp: datetime
```
Note: Passing this single object to the Strategy Engine allows purely mathematical scoring aggregation (e.g., `total_score += state.score * state.confidence`) instead of brittle string matching.

Sector Intelligence (SectorStrengthAnalyzer)
A new asynchronous service powered by `Nifty-50.py` and `Sensex.py` will poll market breadth and constituent strength (e.g., Banking, IT, Auto, Pharma, FMCG).
- **Output Map**: `{"Banking": +8, "IT": +4, "Auto": -2, "FMCG": -6}`
- **Integration**: The `MarketContext` object will pull the most recently cached sector strength scores from memory to augment the Strategy Engine's final bias without blocking the 1-minute execution tick.